UTM
UNIVERSITI TEKNOLOGI MALAYSIA

OBJECT-ORIENTED PROGRAMMING (OOP)
SECJ2154

MINI PROJECT FINAL REPORT

CASE STUDY: ELECTRICITY BILLING SYSTEM

Name	**Matric No**
Lau Yan Kai	A23CS0098
Damiya Aina binti Basir Abd Shammad	A23CS0220
Sabrina Heng Wei Qi	A23CS0265

TABLE OF CONTENTS

1.0 INTRODUCTION

Today, electricity is very important. We need it at home, in shops, and in factories. So, sending bills for electricity is very important too. But using paper or Excel (spreadsheets) to make bills by hand is slow and easy to make mistakes. People may count wrong, send bills late, or get confused when there are too many customers. Also, these old ways can’t change prices during different times of the day or for different usage levels, which can make customers unhappy.

The system is designed to automate the whole billing process from customer registration and consumption tracking to bill calculation, payment handling, and reporting. By using OOP concepts such as classes, inheritance, polymorphism, and encapsulation, the system is modular, reused code, and simple to maintain. This makes the program easier to fix and reuse. For example, customer data becomes an object, and we can use inheritance to make different user types with different powers.

This project shows how using Java and OOP can help make things faster and better. It helps reduce mistakes and makes customers happier.

2.0 PROBLEM STATEMENT

The existing manual electricity billing process presents several challenges. Currently, customer data, metre readings and billing details are often recorded and managed manually using paper-based methods. Some of the major problems identified include:

1. **Inaccuracy in Bill Calculation**
   Manual data entry and calculations increase the risk of incorrect billing, which can lead to disputes and dissatisfaction among customers.

2. **Inefficient Administrative Workflows**
   Preparing bills for multiple users manually is slow, especially in densely populated areas or high-rise residential communities common in Malaysia.

3. **Lack of Organized and Centralized Records**
   Many utility bodies still store customer data in physical files or simple spreadsheets, making it difficult to access, update or analyze billing information quickly.

4. **Difficulty in Managing Tariff Changes**
   Manual systems are not flexible in handling dynamic tariff rates. Any rate change requires manual recalculations and adjustments across all affected bills.

5. **Limited Transparency for Customers**
   Customers do not have access to their past usage or payment history, which limits their ability to monitor electricity consumption or verify bills.

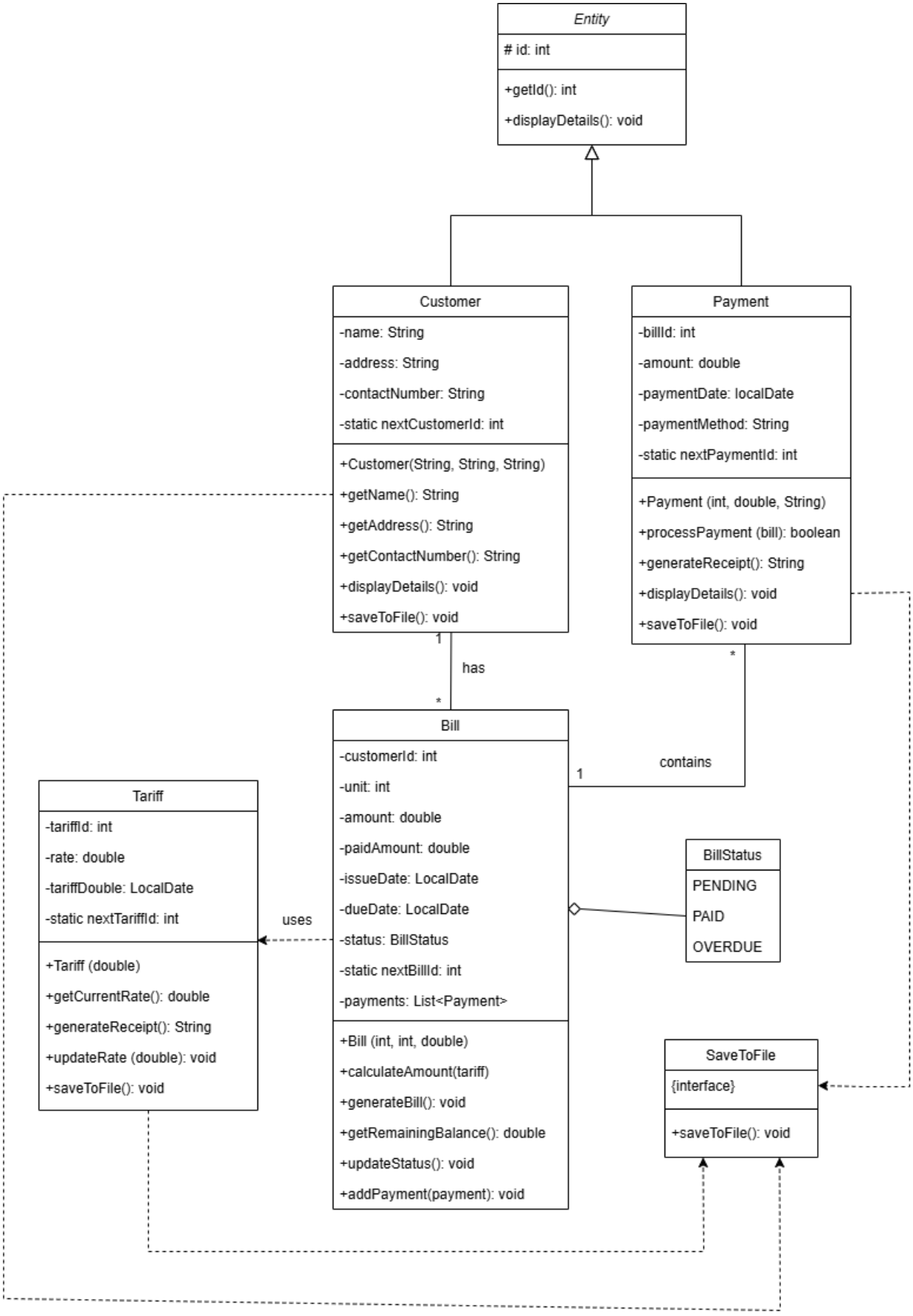
To address these issues, our team proposes an Electricity Billing System using Java and Object-Oriented Programming (OOP) concepts. The system will include key features below:

- Customer Management: Efficiently store and update customer details.
- Bill Generation: Automatically calculate charges based on electricity usage and current tariff rates.
- Tariff Management: Maintain and update electricity rates.
- Payment Processing: Record payments and generate receipts.
- Status Tracking: Use bill status indicators (PENDING, PAID, OVERDUE) for better monitoring.
- File Handling: Save and load customer and billing data.
- Exception Handling: Handle invalid inputs gracefully.

3.0 OBJECTIVES

- To eliminate manual errors in bill calculations
- To digitise customer data management by creating a database system
- To reduce workload by automating bill generation and payment processing
- To enhance billing transparency by providing customers with detailed usage history
- To provide real-time billing status tracking (PAID/PENDING/OVERDUE)
- To demonstrate core OOP principles through modular, reusable Java code architecture

4.0 PROJECT DESIGN (CLASS DIAGRAM)



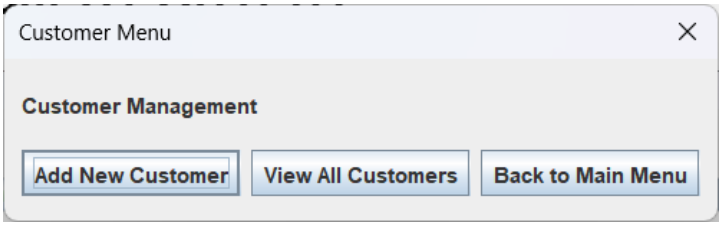
5.0 PROJECT OUTPUT

Screen 1

The main menu of our system – consist of 5 modules

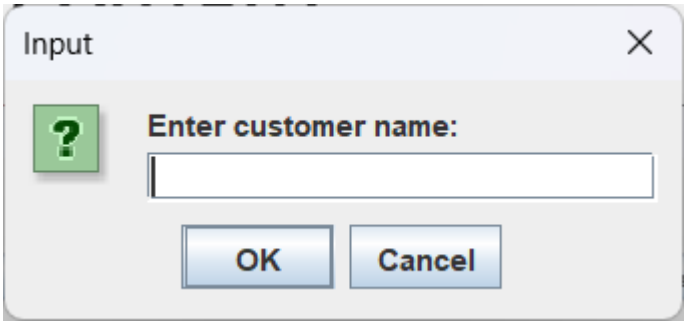
Screen 2

Customer management menu. User can either add new customer, view all customer or back to main menu



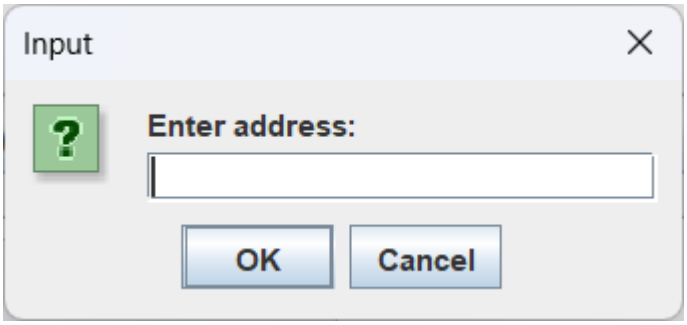
Screen 3

After selecting, add new customer. User need to enter the customer name.

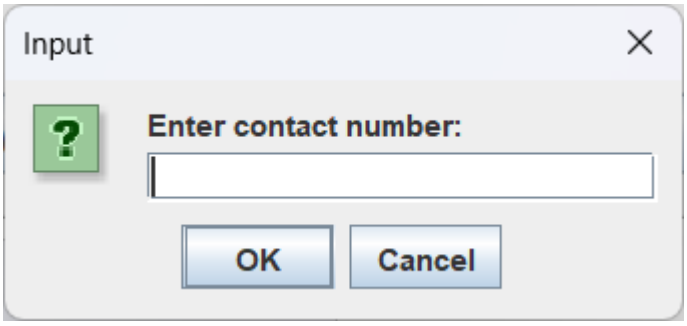


Screen 4

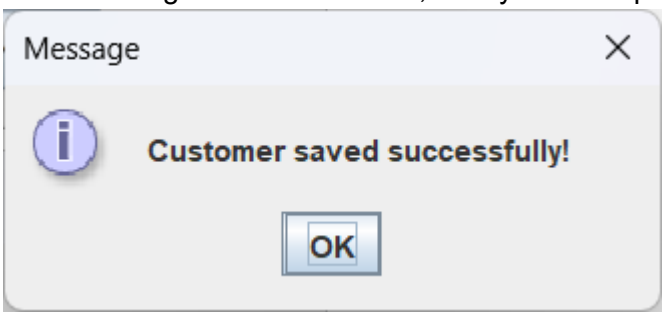
User to enter the address of the new customer.



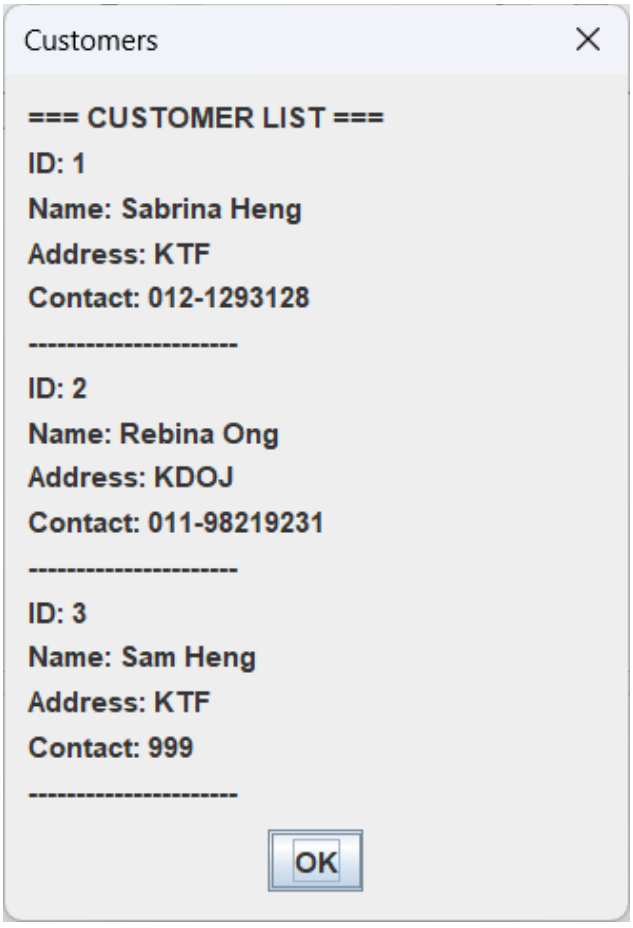
Screen 5
User to enter the contact number of the new customer.



Screen 6
After entering all the information, the system will pop out “Customer saved successfully!”.

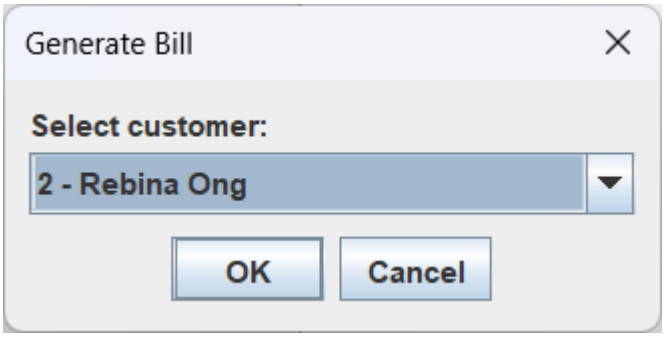


Screen 7
User can select ‘View all customer’ to view the list of customers added in the system.

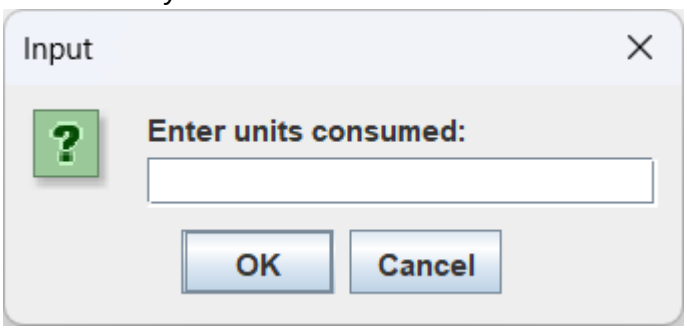


Screen 8
Bill management menu. User can either generate new bill, view all bills or back to main menu

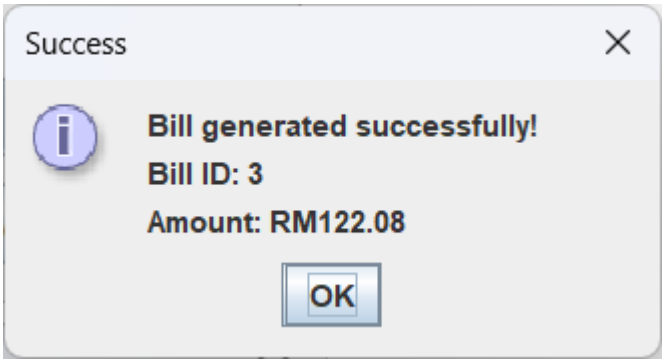
Screen 9
To generate a new bill, user need to select first a customer.



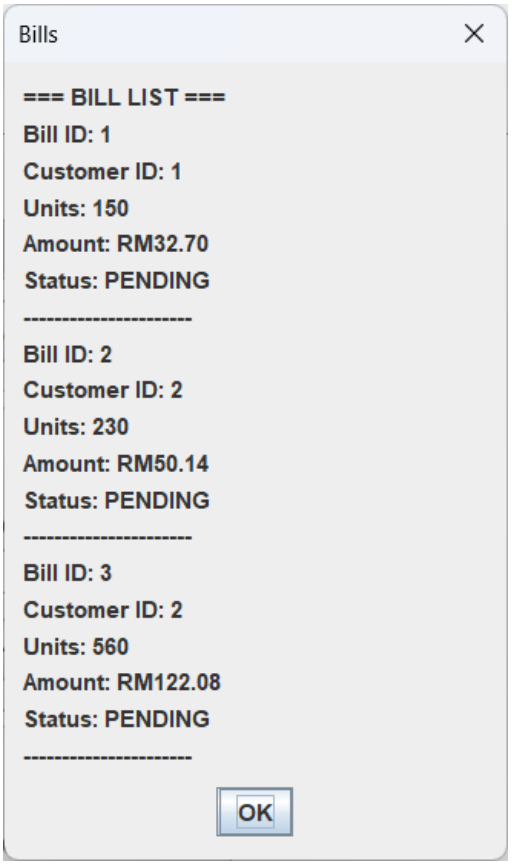
Screen 10
User need to enter the unit consumed for the customer. The bill will be generated based on the unit consumed by the customer.



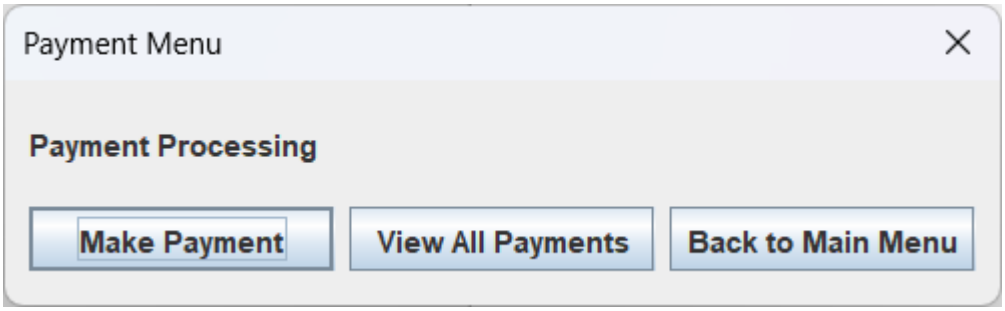
Screen 11
The receipt of bill will be generated once the energy consumption unit is entered. The amount will be displayed.



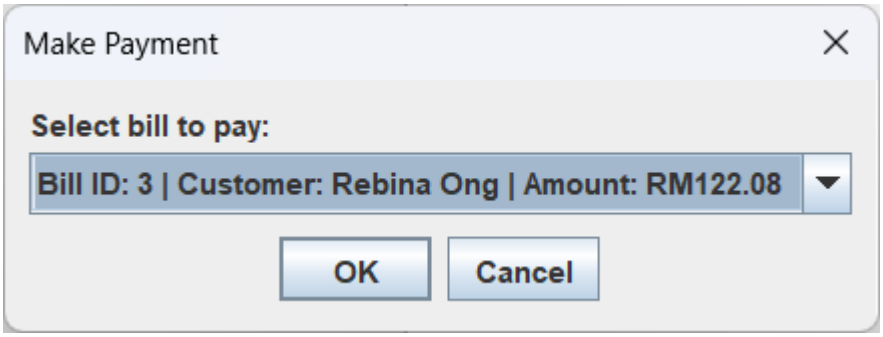
Screen 12
User can select ‘View all bills’ to view the list of bills generated by the system.



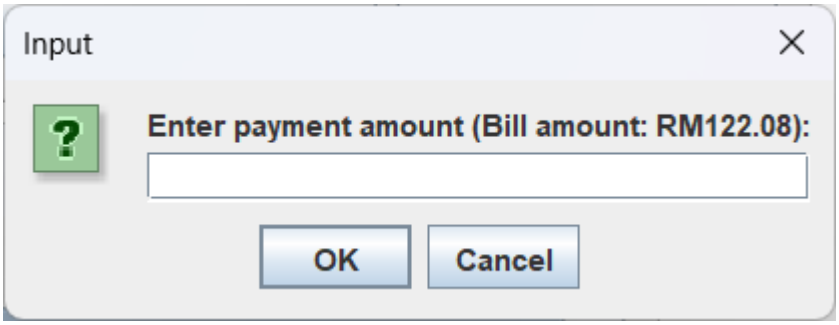
Screen 13
Payment management menu. User can either make payment, view all payments or back to main menu.



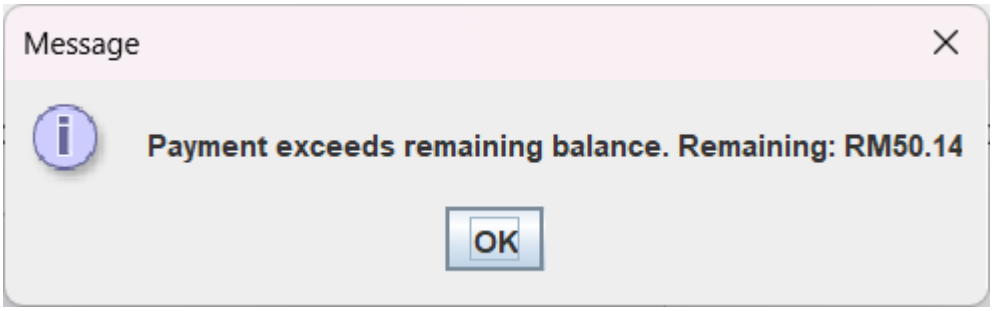
Screen 14
To make payment, user need to select first a customer.



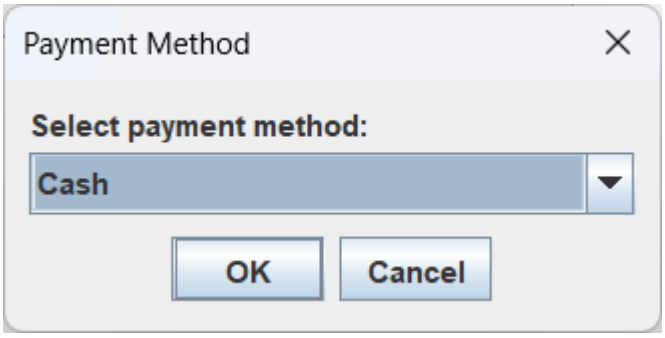
Screen 15
User need to enter the payment amount for the customer. The amount will be deducted from the balance of the bill.



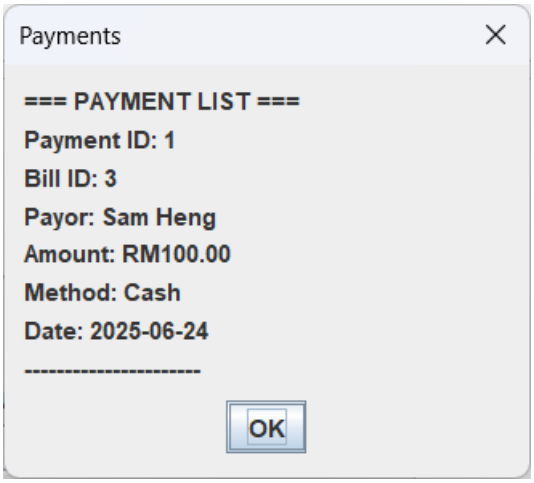
Screen 16
If the user enter the amount that exceeds the remaining balance, it will shows error message.



Screen 17
User can select the payment method, either cash or credit card, or online payment.

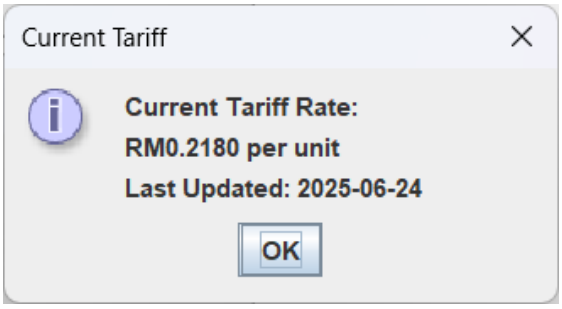


Screen 18
After successful payment, a receipt will be displayed, and user can select View all payments to view the list of payment.

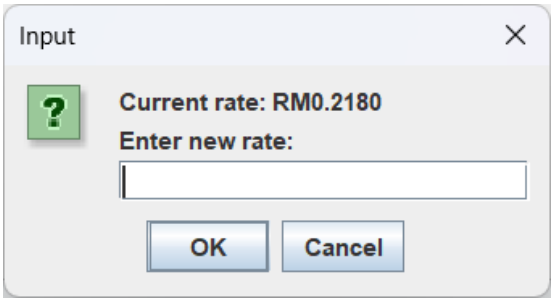


Screen 19
Tariff management menu. User can either view current tariff, update tariff rate or back to main menu.

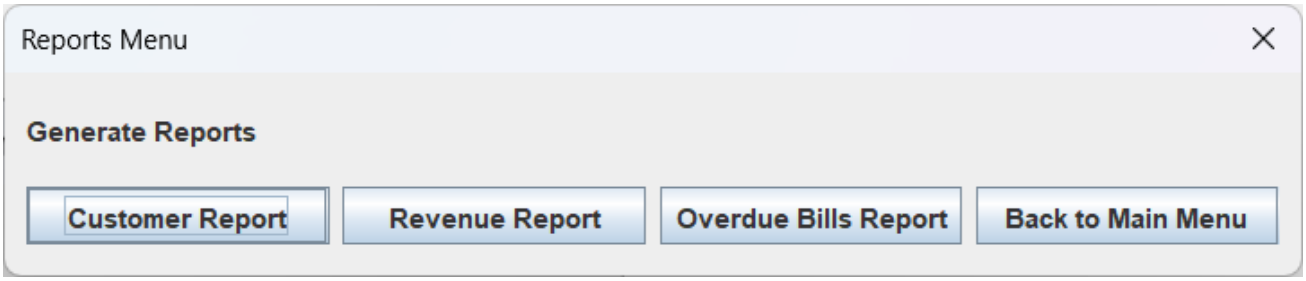
Screen 20
The screen will show the current tariff rate unit price and also the last updated date.



Screen 21
User can enter the new rate for tariff to increase the price of one unit.



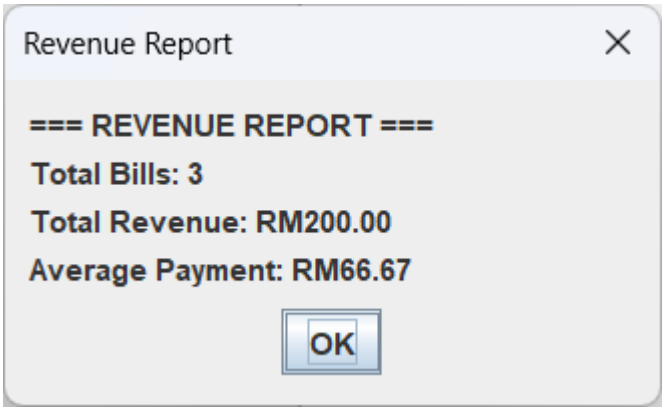
Screen 22
Reports menu. User can either view customer report, revenue report, overdue bills report or back to main menu.



Screen 23
After selecting ‘Customer Report’, it will display the number of customers registered in the system, the ID for each customer, name, address, contact number, total number of bills generated, and also total due amount.

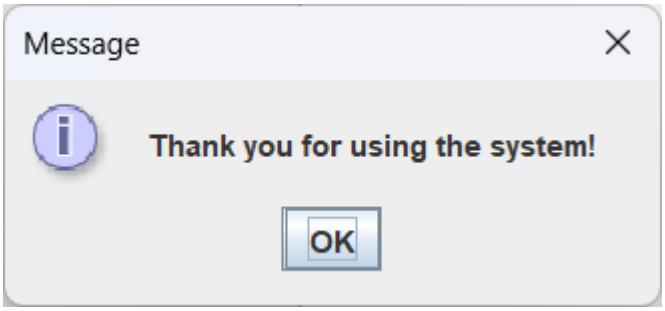
Screen 24

After selecting ‘Revenue Report’, it will display the total number of bills generated by the system, total revenue of the company and average payment amount based on the total number of bills.



Screen 25

The final message by the system after the user select Exit.



6.0 CONCLUSION

This project is called the Electricity Billing System. It helps people and companies get their electricity bills in a faster and more accurate way. Before, people used paper or Excel to make bills by hand. This was slow and caused many mistakes. Now, with this computer system, everything can be done automatically. It saves time and helps workers do their jobs better.

We used Object-Oriented Programming (OOP) to build the system. This is a way of writing code in Java. It uses ideas like Enum, Association, ArrayList, Vector, Composition, Array, Aggregation, Array of Object, Inheritance, File, Static Method, Polymorphism, Abstract class, Interface class, Exception handling and GUI Application. Encapsulation means we hide the inside parts of the code and only show what is needed. Inheritance means one part of the code can use things from another part. Polymorphism means one command can work in different ways. These ideas make the system easier to fix, change, and grow in the future.

While building the system, we faced some problems. We had to learn how to check if the data people enter is correct. This is called data validation. We also learned how to stop the system from crashing when something goes wrong. That’s called exception handling. We saved information in files, but we realized using a database could make it even better for big companies. All these challenges helped us become better at solving problems and writing strong programs.

In the future, we want to make the system even better. We can add a nicer screen for users to click on, connect it to a database to store more data, and let it send messages or reports on its own. Because we used a good design, it will be easy to add these new parts without starting over. This project helped us learn how to turn programming ideas into real working systems. It also gave us confidence to build even bigger software projects one day.

APPENDICES

Bill.java

```java
package ElectricityBillingSystem;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class Bill extends Entity {
    private int customerId;
    private int units;
    private double amount, paidAmount = 0.0;
    private LocalDate issueDate;
    private LocalDate dueDate;
    private BillStatus status;
    private static int nextBillId = 1;
    private List<Payment> payments = new ArrayList<>();

    public Bill(int customerId, int units, Tariff tariff) {
        this.id = nextBillId++;
        this.customerId = customerId;
        this.units = units;
        this.issueDate = LocalDate.now();
        this.dueDate = issueDate.plusDays(30);
        this.status = BillStatus.PENDING;
        calculateAmount(tariff);
    }

    public int getCustomerId() { return customerId; }
    public int getUnits() { return units; }
    public double getAmount() { return amount; }
    public LocalDate getIssueDate() { return issueDate; }
    public LocalDate getDueDate() { return dueDate; }
    public BillStatus getStatus() { return status; }
    public List<Payment> getPayment() { return payments; }

    public void calculateAmount(Tariff tariff) {
        double rate = tariff.getCurrentRate();
        this.amount = units * rate;
    }

    public void generateBill() {
        System.out.println("Bill ID: " + id);
        System.out.println("Customer ID: " + customerId);
        System.out.println("Units Consumed: " + units);
        System.out.println("Amount Due: RM" + String.format("%.2f", amount));
        System.out.println("Issue Date: " + issueDate);
        System.out.println("Due Date: " + dueDate);
        System.out.println("Remaining: RM" + String.format("%.2f",
getRemainingBalance()));
        System.out.println("Status: " + status);
    }

    public double getRemainingBalance() {
        return amount - paidAmount;
    }
```

```
54
55      public void updateStatus() {
56          if (Math.abs(getRemainingBalance()) < 0.01) {
57              status = BillStatus.PAID;
58          } else if (LocalDate.now().isAfter(dueDate)) {
59              status = BillStatus.OVERDUE;
60          } else {
61              status = BillStatus.PENDING;
62          }
63      }
64
65      public void displayDetails() {
66          generateBill();
67      }
68
69      public static int getNextBillId() {
70          return nextBillId;
71      }
72
73      public static void setNextBillId(int nextBillId) {
74          Bill.nextBillId = nextBillId;
75      }
76
77      public void addPayment(Payment payment) {
78          this.paidAmount += payment.getAmount();
79          this.payments.add(payment);
80          updateStatus();
81      }
82  }
```

BillStatus.java

```
1  package ElectricityBillingSystem;
2
3  enum BillStatus {
4      PENDING, PAID, OVERDUE
5  }
```

Customer.java

```
1  package ElectricityBillingSystem;
2  import java.io.FileWriter;
3  import java.io.IOException;
4  import javax.swing.JOptionPane;
5
6  public class Customer extends Entity implements SaveToFile {
7      private String name;
8      private String address;
9      private String contactNumber;
10
11     private static int nextCustomerId = 1;
12
13     public Customer(String name, String address, String contactNumber) {
14         this.id = nextCustomerId++;
15         this.name = name;
16         this.address = address;
17         this.contactNumber = contactNumber;
18     }
19
```

```
    public String getName() { return name; }
    public String getAddress() { return address; }
    public String getContactNumber() { return contactNumber; }

    @Override
    public void displayDetails() {
        System.out.println("Customer ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("Address: " + address);
        System.out.println("Contact Number: " + contactNumber);
    }

    @Override
    public void saveToFile() {
        try {
            FileWriter writer = new FileWriter("customers.txt", true);
            writer.write("Customer ID: " + id + "\n");
            writer.write("Name: " + name + "\n");
            writer.write("Address: " + address + "\n");
            writer.write("Contact Number: " + contactNumber + "\n");
            writer.write("==================================\n");
            writer.close();
            JOptionPane.showMessageDialog(null, "Customer saved
successfully!");
        } catch (IOException e) {
            JOptionPane.showMessageDialog(null, "Error saving customer: " +
e.getMessage());
        }
    }
}
```

Entity.java

```
package ElectricityBillingSystem;
//Abtract Base Class
abstract class Entity {
    protected int id;

    public int getId() { return id; }

    public abstract void displayDetails();
}
```

Payment.java

```
package ElectricityBillingSystem;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import javax.swing.JOptionPane;

public class Payment extends Entity implements SaveToFile {
    private int billID;
    private double amount;
    private LocalDate paymentDate;
    private String paymentMethod;

    private static int nextPaymentID = 1;
```

```
15
16      // Constructor
17      public Payment(int billID, double amount, String paymentMethod) {
18          this.id = nextPaymentID++;
19          this.billID = billID;
20          this.amount = amount;
21          this.paymentDate = LocalDate.now();
22          this.paymentMethod = paymentMethod;
23      }
24
25      // Getter methods
26      public int getPaymentID() { return id; }
27      public int getBillID() { return billID; }
28      public double getAmount() { return amount; }
29      public LocalDate getPaymentDate() { return paymentDate; }
30      public String getPaymentMethod() { return paymentMethod; }
31
32      // Method 1: Process Payment (simple version)
33      public boolean processPayment(Bill bill) {
34          if (bill.getStatus() == BillStatus.PAID) {
35              JOptionPane.showMessageDialog(null, "This bill is already paid
36  in full.");
37              return false;
38          }
39
40          if (amount <= 0) {
41              JOptionPane.showMessageDialog(null, "Payment amount must be
42  positive.");
43              return false;
44          }
45
46          if (amount > bill.getRemainingBalance()) {
47              JOptionPane.showMessageDialog(null,
48                  "Payment exceeds remaining balance. Remaining: RM" +
49                  String.format("%.2f", bill.getRemainingBalance()));
50              return false;
51          }
52
53          bill.addPayment(this);
54          return true;
55      }
56
57
58
59      // Method 2: Generate Receipt
60      public String generateReceipt() {
61          DateTimeFormatter dtf = DateTimeFormatter.ofPattern("yyyy-MM-dd");
62          return "Payment Receipt\n" +
63                 "Payment ID: " + id + "\n" +
64                 "Bill ID: " + billID + "\n" +
65                 "Date: " + dtf.format(paymentDate) + "\n" +
66                 "Method: " + paymentMethod + "\n" +
67                 "Amount: RM" + String.format("%.2f", amount);
68      }
69
70      // Save to file (implements SaveToFile)
71      @Override
72      public void saveToFile() {
```

```
        try {
            FileWriter writer = new FileWriter("payment.txt", true);
            DateTimeFormatter dtf = DateTimeFormatter.ofPattern("yyyy-MM-
dd");

            writer.write("Payment ID: " + id + "\n");
            writer.write("Bill ID: " + billID + "\n");
            writer.write("Payment Date: " + dtf.format(paymentDate) + "\n");
            writer.write("Method: " + paymentMethod + "\n");
            writer.write("Amount: RM" + String.format("%.2f", amount) +
"\n");
            writer.write("==================================\n");
            writer.close();

            JOptionPane.showMessageDialog(null, "Payment saved
successfully!");

        } catch (IOException e) {
            JOptionPane.showMessageDialog(null, "Error saving payment: " +
e.getMessage());
        }
    }

    // Override displayDetails from Entity
    @Override
    public void displayDetails() {
        System.out.println(generateReceipt());
    }
}
```

SaveToFile.java

```
package ElectricityBillingSystem;
// Interface for saving to file
public interface SaveToFile {
    void saveToFile();
}
```

Tariff.java

```
package ElectricityBillingSystem;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import javax.swing.JOptionPane;

public class Tariff {
    private int tariffID;
    private double rate;
    private LocalDate tariffDate;

    private static int nextTariffId = 1;
    public Tariff(double rate) {
        this.tariffID = nextTariffId++;
        this.rate = rate;
        this.tariffDate = LocalDate.now();
    }

    public double getCurrentRate() {
```

```
        return rate;
    }

    public int getTariffID() {
        return tariffID;
    }

    public LocalDate getTariffDate() {
        return tariffDate;
    }

    public void updateRate(double newRate) {
        this.rate = newRate;
        this.tariffDate = LocalDate.now();
    }

    public void displayDetails() {
        System.out.println("Tariff ID: " + tariffID);
        System.out.println("Rate: RM" + String.format("%.2f", rate));
        System.out.println("Updated Date: " + tariffDate);
    }

    public void saveToFile() {
        try {
            FileWriter writer = new FileWriter("tariff.txt", true);
            writer.write("Tariff ID: " + tariffID + "\n");
            writer.write("Rate: RM" + String.format("%.2f", rate) + "\n");
            writer.write("Updated Date: " + tariffDate + "\n");
            writer.write("==================================\n");
            writer.close();
            JOptionPane.showMessageDialog(null, "Tariff saved
successfully!");
        } catch (IOException e) {
            JOptionPane.showMessageDialog(null, "Error saving tariff: " +
e.getMessage());
        }
    }
}
```

ElectricityBillingSystem.java

```
/*
! Electricity Billing System

Mini Group Project

---

Project Title:
*Electricity Billing System*

Course:
Object-Oriented Programming (OOP)

Group Members:
1.Sabrina Heng Wei Qi
2. Damiya Aina binti Basir Abd Shammad
3. Lau Yan Kai
```

```
18
19  Lecturer:
20  Dr Zuraini Ali Shah
21
22  Submission Date:
23  24 June 2025
24
25  ---
26
27  Project Description:
28
29  This mini project is developed to simulate a simple *Electricity Billing
30  System* using Java programming with Object-Oriented Programming (OOP)
31  concepts. The system allows managing customer information, tariff rates, and
32  saving data into files. It applies OOP principles such as encapsulation,
33  inheritance, polymorphism, abstraction, and exception handling.
34
35
36  Features Implemented:
37
38  - Customer registration with auto-generated Customer ID
39  - Tariff rate management with update and save functionality
40  - File storage for customer and tariff data
41  - Simple console and pop-up interface
42
43   */
44
45
46  package ElectricityBillingSystem;
47
48  import javax.swing.JOptionPane;
49  import java.util.ArrayList;
50  import java.util.List;
51  import java.time.LocalDate;
52
53  public class ElectricityBillingSystem {
54      private static List<Customer> customers = new ArrayList<>();
55      private static List<Bill> bills = new ArrayList<>();
56      private static List<Payment> payments = new ArrayList<>();
57      private static Tariff currentTariff = new Tariff(0.218); // Default rate
58
59      public static void main(String[] args) {
60          loadSampleData();
61
62          while (true) {
63              String[] options = {
64                  "Customer Management",
65                  "Bill Management",
66                  "Payment Processing",
67                  "Tariff Management",
68                  "Reports",
69                  "Exit"
70              };
71
72              int choice = JOptionPane.showOptionDialog(
73                  null,
74                  "Electricity Billing System\nSelect an option:",
75                  "Main Menu",
```

```
                JOptionPane.DEFAULT_OPTION,
                JOptionPane.PLAIN_MESSAGE,
                null,
                options,
                options[0]
            );

            try {
                switch (choice) {
                    case 0: customerManagement(); break;
                    case 1: billManagement(); break;
                    case 2: paymentProcessing(); break;
                    case 3: tariffManagement(); break;
                    case 4: generateReports(); break;
                    case 5:
                        JOptionPane.showMessageDialog(null, "Thank you for
using the system!");
                        System.exit(0);
                    default:
                        if (choice == -1) System.exit(0); // Close button
                }
            } catch (Exception e) {
                JOptionPane.showMessageDialog(null, "Error: " +
e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
            }
        }
    }

    private static void customerManagement() {
        String[] options = {
            "Add New Customer",
            "View All Customers",
            "Back to Main Menu"
        };

        int choice = JOptionPane.showOptionDialog(
            null,
            "Customer Management",
            "Customer Menu",
            JOptionPane.DEFAULT_OPTION,
            JOptionPane.PLAIN_MESSAGE,
            null,
            options,
            options[0]
        );

        switch (choice) {
            case 0: addCustomer(); break;
            case 1: viewAllCustomers(); break;
            case 2: return;
            default: return;
        }
    }

    private static void addCustomer() {
        String name = JOptionPane.showInputDialog("Enter customer name:");
        if (name == null || name.trim().isEmpty()) return;

```

```
134         String address = JOptionPane.showInputDialog("Enter address:");
135         if (address == null || address.trim().isEmpty()) return;
136 
137         String contact = JOptionPane.showInputDialog("Enter contact
138 number:");
139         if (contact == null || contact.trim().isEmpty()) return;
140 
141         Customer customer = new Customer(name, address, contact);
142         customers.add(customer);
143         customer.saveToFile();
144 
145         JOptionPane.showMessageDialog(null,
146             "Customer added successfully!\nID: " + customer.getId(),
147             "Success", JOptionPane.INFORMATION_MESSAGE);
148     }
149 
150     private static void viewAllCustomers() {
151         if (customers.isEmpty()) {
152             JOptionPane.showMessageDialog(null, "No customers found.");
153             return;
154         }
155 
156         StringBuilder sb = new StringBuilder();
157         sb.append("=== CUSTOMER LIST ===\n");
158         for (Customer c : customers) {
159             sb.append("ID: ").append(c.getId()).append("\n");
160             sb.append("Name: ").append(c.getName()).append("\n");
161             sb.append("Address: ").append(c.getAddress()).append("\n");
162             sb.append("Contact:
163 ").append(c.getContactNumber()).append("\n");
164             sb.append("----------------------\n");
165         }
166 
167         JOptionPane.showMessageDialog(null, sb.toString(), "Customers",
168 JOptionPane.PLAIN_MESSAGE);
169     }
170 
171     private static void billManagement() {
172         String[] options = {
173             "Generate New Bill",
174             "View All Bills",
175             "Back to Main Menu"
176         };
177 
178         int choice = JOptionPane.showOptionDialog(
179             null,
180             "Bill Management",
181             "Bill Menu",
182             JOptionPane.DEFAULT_OPTION,
183             JOptionPane.PLAIN_MESSAGE,
184             null,
185             options,
186             options[0]
187         );
188 
189         switch (choice) {
190             case 0: generateBill(); break;
191             case 1: viewAllBills(); break;
```

```
            case 2: return;
            default: return;
        }
    }

    private static void generateBill() {
        if (customers.isEmpty()) {
            JOptionPane.showMessageDialog(null, "No customers available.
Please add customers first.");
            return;
        }

        // Create customer selection list
        String[] customerOptions = new String[customers.size()];
        for (int i = 0; i < customers.size(); i++) {
            customerOptions[i] = customers.get(i).getId() + " - " +
customers.get(i).getName();
        }

        String selectedCustomer = (String) JOptionPane.showInputDialog(
            null,
            "Select customer:",
            "Generate Bill",
            JOptionPane.PLAIN_MESSAGE,
            null,
            customerOptions,
            customerOptions[0]
        );

        if (selectedCustomer == null) return;

        int customerId = Integer.parseInt(selectedCustomer.split(" - ")[0]);
        Customer customer = customers.stream()
            .filter(c -> c.getId() == customerId)
            .findFirst()
            .orElse(null);

        String unitsStr = JOptionPane.showInputDialog("Enter units
consumed:");
        if (unitsStr == null || unitsStr.trim().isEmpty()) return;

        try {
            int units = Integer.parseInt(unitsStr);
            Bill bill = new Bill(customer.getId(), units, currentTariff);
            bills.add(bill);

            JOptionPane.showMessageDialog(null,
                "Bill generated successfully!\n" +
                "Bill ID: " + bill.getId() + "\n" +
                "Amount: RM" + String.format("%.2f", bill.getAmount()),
                "Success", JOptionPane.INFORMATION_MESSAGE);
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(null, "Invalid units entered.",
"Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    private static void viewAllBills() {
```

```
250         if (bills.isEmpty()) {
251             JOptionPane.showMessageDialog(null, "No bills found.");
252             return;
253         }
254 
255         StringBuilder sb = new StringBuilder();
256         sb.append("=== BILL LIST ===\n");
257         for (Bill b : bills) {
258             sb.append("Bill ID: ").append(b.getId()).append("\n");
259             sb.append("Customer ID:
260 ").append(b.getCustomerId()).append("\n");
261             sb.append("Units: ").append(b.getUnits()).append("\n");
262             sb.append("Amount: RM").append(String.format("%.2f",
263 b.getAmount())).append("\n");
264             sb.append("Status: ").append(b.getStatus()).append("\n");
265             sb.append("----------------------\n");
266         }
267 
268         JOptionPane.showMessageDialog(null, sb.toString(), "Bills",
269 JOptionPane.PLAIN_MESSAGE);
270     }
271 
272     private static void paymentProcessing() {
273         String[] options = {
274             "Make Payment",
275             "View All Payments",
276             "Back to Main Menu"
277         };
278 
279         int choice = JOptionPane.showOptionDialog(
280             null,
281             "Payment Processing",
282             "Payment Menu",
283             JOptionPane.DEFAULT_OPTION,
284             JOptionPane.PLAIN_MESSAGE,
285             null,
286             options,
287             options[0]
288         );
289 
290         switch (choice) {
291             case 0: makePayment(); break;
292             case 1: viewAllPayments(); break;
293             case 2: return;
294             default: return;
295         }
296     }
297 
298     private static void makePayment() {
299         if (bills.isEmpty()) {
300             JOptionPane.showMessageDialog(null, "No bills available for
301 payment.");
302             return;
303         }
304 
305         // Create bill selection list (only unpaid bills)
306         List<Bill> unpaidBills = new ArrayList<>();
307         for (Bill b : bills) {
```

```
            if (b.getStatus() != BillStatus.PAID) {
                unpaidBills.add(b);
            }
        }

        if (unpaidBills.isEmpty()) {
            JOptionPane.showMessageDialog(null, "No unpaid bills
available.");
            return;
        }

        String[] billOptions = new String[unpaidBills.size()];
        for (int i = 0; i < unpaidBills.size(); i++) {
            Bill b = unpaidBills.get(i);
            Customer customer = findCustomerById(b.getCustomerId());
            String customerName = (customer != null) ? customer.getName() :
"Unknown";
            billOptions[i] = String.format("Bill ID: %d | Customer: %s |
Amount: RM%.2f",
                                           b.getId(), customerName,
b.getRemainingBalance());
        }

        String selectedBill = (String) JOptionPane.showInputDialog(
            null,
            "Select bill to pay:",
            "Make Payment",
            JOptionPane.PLAIN_MESSAGE,
            null,
            billOptions,
            billOptions[0]
        );

        if (selectedBill == null) return;

        int billId =
Integer.parseInt(selectedBill.split("\\|")[0].replaceAll("[^0-9]",
"").trim());

        Bill bill = bills.stream()
            .filter(b -> b.getId() == billId)
            .findFirst()
            .orElse(null);

        String amountStr = JOptionPane.showInputDialog(
            "Enter payment amount (Bill amount: RM" + String.format("%.2f",
bill.getRemainingBalance()) + "):");
        if (amountStr == null || amountStr.trim().isEmpty()) return;

        String[] methods = {"Cash", "Credit Card", "Online Transfer"};
        String method = (String) JOptionPane.showInputDialog(
            null,
            "Select payment method:",
            "Payment Method",
            JOptionPane.PLAIN_MESSAGE,
            null,
            methods,
            methods[0]
```

```
366             );
367
368             if (method == null) return;
369
370             try {
371             double amount = Double.parseDouble(amountStr);
372             Payment payment = new Payment(bill.getId(), amount, method);
373                 if (payment.processPayment(bill)) {
374                     payments.add(payment);
375                     bill.addPayment(payment);
376                     payment.saveToFile();
377                     bill.updateStatus();
378
379                     JOptionPane.showMessageDialog(null,
380                         "Payment processed successfully!\n" +
381                         "Amount Paid: RM" + String.format("%.2f", amount) + "\n"
382 +
383                         payment.generateReceipt(),
384                         "Success", JOptionPane.INFORMATION_MESSAGE);
385                 }
386             } catch (NumberFormatException e) {
387                 JOptionPane.showMessageDialog(null, "Invalid amount entered.",
388 "Error", JOptionPane.ERROR_MESSAGE);
389             }
390
391     }
392
393     private static void viewAllPayments() {
394             if (payments.isEmpty()) {
395                 JOptionPane.showMessageDialog(null, "No payments found.");
396                 return;
397             }
398
399             StringBuilder sb = new StringBuilder();
400             sb.append("=== PAYMENT LIST ===\n");
401             for (Payment p : payments) {
402                 sb.append("Payment ID: ").append(p.getPaymentID()).append("\n");
403                 sb.append("Bill ID: ").append(p.getBillID()).append("\n");
404                 sb.append(null == findCustomerById(p.getBillID()) ? "Payor:
405 Unknown" : "Payor: " +
406 findCustomerById(p.getBillID()).getName()).append("\n");
407                 sb.append("Amount: RM").append(String.format("%.2f",
408 p.getAmount())).append("\n");
409                 sb.append("Method: ").append(p.getPaymentMethod()).append("\n");
410                 sb.append("Date: ").append(p.getPaymentDate()).append("\n");
411                 sb.append("----------------------\n");
412             }
413
414             JOptionPane.showMessageDialog(null, sb.toString(), "Payments",
415 JOptionPane.PLAIN_MESSAGE);
416     }
417
418     private static void tariffManagement() {
419             String[] options = {
420                 "View Current Tariff",
421                 "Update Tariff Rate",
422                 "Back to Main Menu"
423             };
```

```
424
425         int choice = JOptionPane.showOptionDialog(
426             null,
427             "Tariff Management",
428             "Tariff Menu",
429             JOptionPane.DEFAULT_OPTION,
430             JOptionPane.PLAIN_MESSAGE,
431             null,
432             options,
433             options[0]
434         );
435
436         switch (choice) {
437             case 0: viewCurrentTariff(); break;
438             case 1: updateTariffRate(); break;
439             case 2: return;
440             default: return;
441         }
442     }
443
444     private static void viewCurrentTariff() {
445         JOptionPane.showMessageDialog(null,
446             "Current Tariff Rate:\nRM" + String.format("%.4f",
447 currentTariff.getCurrentRate()) + " per unit\n" +
448             "Last Updated: " + currentTariff.getTariffDate(),
449             "Current Tariff", JOptionPane.INFORMATION_MESSAGE);
450     }
451
452     private static void updateTariffRate() {
453         String newRateStr = JOptionPane.showInputDialog(
454             "Current rate: RM" + String.format("%.4f",
455 currentTariff.getCurrentRate()) + "\n" +
456             "Enter new rate:");
457
458         if (newRateStr == null || newRateStr.trim().isEmpty()) return;
459
460         try {
461             double newRate = Double.parseDouble(newRateStr);
462             currentTariff.updateRate(newRate);
463             currentTariff.saveToFile();
464
465             JOptionPane.showMessageDialog(null,
466                 "Tariff rate updated successfully!\nNew rate: RM" +
467 String.format("%.4f", currentTariff.getCurrentRate()),
468                 "Success", JOptionPane.INFORMATION_MESSAGE);
469         } catch (NumberFormatException e) {
470             JOptionPane.showMessageDialog(null, "Invalid rate entered.",
471 "Error", JOptionPane.ERROR_MESSAGE);
472         }
473     }
474
475     private static void generateReports() {
476         String[] options = {
477             "Customer Report",
478             "Revenue Report",
479             "Overdue Bills Report",
480             "Back to Main Menu"
481         };
```

```
482
483         int choice = JOptionPane.showOptionDialog(
484             null,
485             "Generate Reports",
486             "Reports Menu",
487             JOptionPane.DEFAULT_OPTION,
488             JOptionPane.PLAIN_MESSAGE,
489             null,
490             options,
491             options[0]
492         );
493
494         switch (choice) {
495             case 0: customerReport(); break;
496             case 1: revenueReport(); break;
497             case 2: overdueBillsReport(); break;
498             case 3: return;
499             default: return;
500         }
501     }
502
503     private static void customerReport() {
504         if (customers.isEmpty()) {
505             JOptionPane.showMessageDialog(null, "No customers found.");
506             return;
507         }
508
509         StringBuilder sb = new StringBuilder();
510         sb.append("=== CUSTOMER REPORT ===\n");
511         sb.append("Total Customers:
512 ").append(customers.size()).append("\n\n");
513
514         for (Customer c : customers) {
515             sb.append("ID: ").append(c.getId()).append("\n");
516             sb.append("Name: ").append(c.getName()).append("\n");
517             sb.append("Address: ").append(c.getAddress()).append("\n");
518             sb.append("Contact:
519 ").append(c.getContactNumber()).append("\n");
520
521             // Calculate total bills and amount due
522             long billCount = bills.stream().filter(b -> b.getCustomerId() ==
523 c.getId()).count();
524             double totalDue = bills.stream()
525                 .filter(b -> b.getCustomerId() == c.getId() &&
526 b.getStatus() != BillStatus.PAID)
527                 .mapToDouble(Bill::getAmount)
528                 .sum();
529
530             sb.append("Total Bills: ").append(billCount).append("\n");
531             sb.append("Total Due: RM").append(String.format("%.2f",
532 totalDue)).append("\n");
533             sb.append("-----------------------\n");
534         }
535
536         JOptionPane.showMessageDialog(null, sb.toString(), "Customer
537 Report", JOptionPane.PLAIN_MESSAGE);
538     }
539
```

```
    private static void revenueReport() {
        // Calculate total revenue from all payments for paid bills
        double totalRevenue = bills.stream()
            .flatMap(b -> b.getPayment().stream())
            .mapToDouble(Payment::getAmount)
            .sum();

        int paidBills = (int) bills.stream()
            .count();

        StringBuilder sb = new StringBuilder();
        sb.append("=== REVENUE REPORT ===\n");
        sb.append("Total Bills: ").append(paidBills).append("\n");
        sb.append("Total Revenue: RM").append(String.format("%.2f",
totalRevenue)).append("\n");

        if (paidBills > 0) {
            sb.append("Average Payment: RM")
            .append(String.format("%.2f",
totalRevenue/paidBills)).append("\n");
        } else {
            sb.append("Average Payment: RM0.00\n");
        }

        JOptionPane.showMessageDialog(null, sb.toString(), "Revenue Report",
JOptionPane.PLAIN_MESSAGE);
    }

    private static void overdueBillsReport() {
        long overdueCount = bills.stream().filter(b -> b.getStatus() ==
BillStatus.OVERDUE).count();
        double totalOverdue = bills.stream()
            .filter(b -> b.getStatus() == BillStatus.OVERDUE)
            .mapToDouble(Bill::getAmount)
            .sum();

        if (overdueCount == 0) {
            JOptionPane.showMessageDialog(null, "No overdue bills found.");
            return;
        }

        StringBuilder sb = new StringBuilder();
        sb.append("=== OVERDUE BILLS REPORT ===\n");
        sb.append("Total Overdue Bills:
").append(overdueCount).append("\n");
        sb.append("Total Overdue Amount: RM").append(String.format("%.2f",
totalOverdue)).append("\n\n");

        bills.stream()
            .filter(b -> b.getStatus() == BillStatus.OVERDUE)
            .forEach(b -> {
                sb.append("Bill ID: ").append(b.getId()).append("\n");
                sb.append("Customer ID:
").append(b.getCustomerId()).append("\n");
                sb.append("Amount: RM").append(String.format("%.2f",
b.getAmount())).append("\n");
                sb.append("Due Date: ").append(b.getDueDate()).append("\n");

```

```
                sb.append("Days Overdue:
").append(LocalDate.now().until(b.getDueDate()).getDays()).append("\n");
                sb.append("------------------------\n");
            });

        JOptionPane.showMessageDialog(null, sb.toString(), "Overdue Bills
Report", JOptionPane.PLAIN_MESSAGE);
    }

    private static Customer findCustomerById(int customerId) {
        for (Customer customer : customers) {
            if (customer.getId() == customerId) {
                return customer;
            }
        }
        return null;
    }

    private static void loadSampleData() {
        // Sample customers
        customers.add(new Customer("Sabrina Heng", "KTF", "012-1293128"));
        customers.add(new Customer("Rebina Ong", "KDOJ", "011-98219231"));

        // Sample bills
        bills.add(new Bill(1, 150, currentTariff));
        bills.add(new Bill(2, 230, currentTariff));
    }
}
```